TEAM FIELD GUIDE

First-Time Team Git & Collaboration

Operating Standard for 2-Developer Projects (Frontend + Backend Lead)

Target: Zero Code Losses
Workflow: Feature Branch PRs

1. THE 2-DEVELOPER COLLABORATION MINDSET

Working in a team requires a shift from single-player coding to **shared ownership**. When two developers work simultaneously, code changes can easily overwrite each other if not coordinated through structured version control procedures.

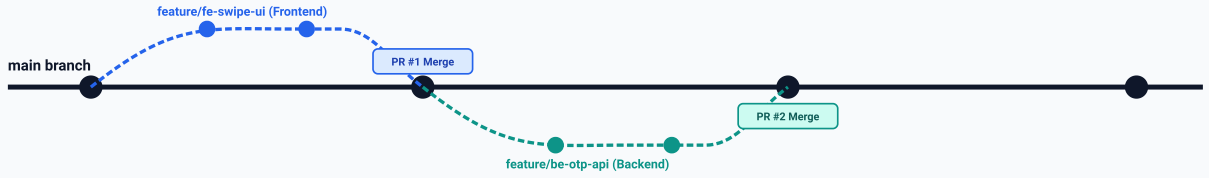
Rule #1: Main is Sacred

The `main` branch represents working, production-ready code. Never write code directly on `main`. All work happens on dedicated feature branches.

Rule #2: Sync Before You Code

Always pull the latest changes from `main` before starting a new feature branch to ensure you are building on top of your partner's latest work.

2. FEATURE BRANCH WORKFLOW VISUAL ARCHITECTURE



3. RECOMMENDED REPOSITORY STRUCTURE (MONOREPO)

Keep both client and server code inside a single GitHub repository for simplicity during initial development:

```
skill-barter-platform/
├── client/ # React / Next.js app (Frontend Lead)
├── server/ # Node.js / Express API (Backend Lead)
├── .gitignore # Global ignore rules
└── README.md # Project setup instructions
```

4. THE DAILY GIT LIFECYCLE (STEP-BY-STEP)

1. **Start of Day:** Switch to `main` and fetch the latest changes committed by your partner.
   `git checkout main` & `git pull origin main`
2. **Create Feature Branch:** Branch out with a descriptive prefix.
   `git checkout -b feature/fe-swipe-card` or `feature/be-swipe-api`
3. **Work & Commit Frequently:** Make small, logical commits as you build.
   `git add .` & `git commit -m "add swipe card drag gestures"`
4. **Push Branch to GitHub:** Upload your branch for review.
   `git push origin feature/fe-swipe-card`
5. **Open Pull Request (PR):** Create a PR on GitHub targetting `main`.
6. **Peer Review & Merge:** Your partner reviews the code, approves it, and clicks **Merge**.

5. ESSENTIAL TEAM GIT COMMAND CHEATSHEET

Goal	Command	When to Use
Get Latest Code	`git pull origin main`	Every morning before starting work.
New Feature Branch	`git checkout -b feature/name`	Before starting any new task.
Check Status	`git status`	To see edited or unstaged files.
Stage Changes	`git add .`	Before committing code updates.
Commit Work	`git commit -m "msg"`	When a logical sub-task is completed.
Push to GitHub	`git push origin feature/name`	When ready for PR code review.
Switch Branches	`git checkout branch-name`	Moving between feature branches.
Discard Unsaved Edits	`git restore .`	Undoing local mistakes safely.

6. PULL REQUEST (PR) & CODE REVIEW CHECKLIST

Frontend PR Checklist

- Includes API endpoint payload model alignment.
- Tested responsiveness on mobile & desktop views.
- No hardcoded mock URLs (uses `process.env`).

Backend PR Checklist

- Database migration scripts included if schema changed.
- API returns clear HTTP status codes (200, 400, 401).
- Validated input fields against missing or bad parameters.

7. HANDLING MERGE CONFLICTS (DON'T PANIC!)

A **Merge Conflict** occurs when both developers edit the same line of the same file. Git pauses and asks you to choose which version to keep.

```
# Conflict Markers inside file:
<<<<<<< HEAD (Your Current Branch Changes)
const API_URL = "http://localhost:5000/api";
=======
const API_URL = "https://api.skillbarter.com";
>>>>>>> main (Partner's Incoming Changes)
```

Step-by-step resolution:

1. Open the conflicted file in VS Code. VS Code highlights both options with "Accept Current Change" or "Accept Incoming Change" buttons.
2. Discuss with your partner which line is correct, delete the marker lines, and save the file.
3. Run `git add .` followed by `git commit -m "fix merge conflict"` and push.

8. ENVIRONMENT SECRETS & `.GITIGNORE` RULES

Protecting Sensitive Keys

NEVER commit passwords, API keys, database URLs, or `.env` files to GitHub. Public leaks can compromise your database or incur unexpected Twilio/AWS charges.

Always ignore:

- `.env` & `.env.local`
- `node_modules/`
- `.DS_Store` / `dist/`

Pattern to share config:

Commit a dummy template named `.env.example` with placeholder values so your teammate knows what variables are required.

9. GOLDEN RULES OF TEAM GIT COLLABORATION

✔ ALWAYS DO	✖ NEVER DO
Pull `main` before starting a new feature branch.	Never push directly to the `main` branch.
Write clear commit messages (e.g. "add user profile API").	Never write vague messages like "fixed stuff" or "asdasd".
Keep PRs small so they are fast and easy to review.	Never submit a giant PR editing 50 files at once.
Notify your partner on Discord/WhatsApp when a PR is open.	Never force push (`git push -f`) to shared branches.